

Supplementary Material: When Learning Paths Lie – Gradient Retention under Hidden Relations

Anonymous submission

Environment Details

510K Card Game

510K is a four-player trick-taking game with a standard 52-card deck. Cards 5, 10, K carry point values (5, 10, 10). The lead player sets the pattern; others must follow with a higher-rank play or pass. The trick winner collects all 5/10/K points.

Action space: 300 discrete actions (0 = pass, 1–299 = pattern index). **Observation:** 112-dim Box (116-dim for OBVIOUS mode, with 4 additional teammate-indicator bits). **Pattern types (15):** SINGLE, PAIR, THREE, THREE_ONE, THREE_PAIR, STRAIGHT (5+), CONSECUTIVE_PAIRS (3+), AIRPLANE_NONE, AIRPLANE_SINGLE, AIRPLANE_PAIR, BOMB, NONSUIT_510K, SUIT_510K, RED_A_SINGLE, RED_A_PAIR.

Four modes share identical card mechanics but differ in team structure and termination:

- **SINGLE:** solo-competitive. Game ends when first player empties hand. Winner +30, losers −10 each (plus own 510K point accumulation).
- **STATIC:** fixed teams (players 0&2 vs. 1&3). Both teammates must finish. Winners receive a $\times 2$ score multiplier; losers $\times 1$.
- **DYNAMIC:** teams determined by hidden red-Ace distribution (holders of $A\heartsuit/A\spadesuit$ form one team). Same scoring as STATIC. Red A is the strongest single; red-A pair is the strongest bomb. Agent must infer teammates from gameplay—team membership is never observed.
- **OBVIOUS:** identical card mechanics to DYNAMIC, but team membership is observable as a 4-bit observation. This isolates *information structure* from card mechanics.

OBVIOUS construction. The OBVIOUS observation is 116-dimensional (112 base + 4 teammate-indicator bits). Bit $i = 1$ if player i is on the agent’s team. This is the only structural difference from DYNAMIC (112-dim). During continuous reveal experiments, these bits are randomly masked

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

per-timestep with probability $1 - p$ where p is the reveal fraction. Per-timestep masking was chosen over per-episode masking to model a noisy sensor that occasionally fails—a realistic information-access pattern in deployed systems.

Behavioral features for path integral. Seven mode-independent features (ϕ_1 – ϕ_7), all with VIF < 2 :

- ϕ_1 **MyScore:** cumulative 510K points, normalized by 150.
- ϕ_2 **MyHandSize:** fraction of hand played, $(13 - |h|)/13$.
- ϕ_3 **MyStrength:** average card rank, mapped to $[0, 1]$.
- ϕ_4 **TrickScore:** points at stake in current trick, normalized by 100.
- ϕ_5 **PassCount:** consecutive passes in current trick, normalized.
- ϕ_6 **ScoreSpread:** std. of scores across players, normalized by 50. (Cooperative.)
- ϕ_7 **SuppressionGap:** rank gap to leader’s strongest card, normalized by 12. (Cooperative.)

Feature ablation: deleting any single feature preserves the path-length ordering SINGLE $>$ STATIC $>$ DYNAMIC (7/7 subsets).

Toy Matching Environment

A minimal 2-action contextual bandit that isolates IIGC:

- **State:** 1-dim (constant 0) in HIDDEN; 2-dim (one-hot partner type) in REVEALED.
- **Action:** $a \in \{0, 1\}$. Agent chooses which action to execute.
- **Partner:** hidden binary type $p \in \{0, 1\}$, fixed per episode.
- **Reward:** +1 if $a = p$, −1 if $a \neq p$.
- **Training:** 10K steps, 10 independent seeds per condition.

In HIDDEN mode, the agent receives no information about p . Gradients from episodes where $p = 0$ and $p = 1$ cancel nearly completely ($\kappa \approx 0$). In REVEALED mode, the agent observes p directly and learns the optimal type-matching policy.

Overcooked Environment

Grid-world cooperative cooking game with configurable partner roles:

- **State:** 96-dim observation (grid layout, object positions, agent inventory).

- **Action:** 6 discrete actions (up/down/left/right/interact/stay).
- **Layout:** cramped_room (standard Overcooked layout).
- **Reward:** sparse +20 per soup delivered, −0.01 per timestep.
- **Horizon:** 400 steps per episode.
- **Training:** PPO, 1M steps, 8 parallel envs, 8 independent seeds per mode.

Partner agent design. Three specialized partner agents create genuine strategic divergence. The key design principle is **role complementarity**: chef and waiter perform mutually exclusive subtasks. A policy optimized for one role (e.g., delivering when paired with a chef) fails entirely under the other role (delivering when paired with a waiter, where nobody cooks). This ensures that hidden-role training faces genuine gradient conflict.

- **Chef** (cook): Picks up onions from supply depots and places them in empty or partially-filled pots. Never picks up plates or delivers finished soups. Implements a greedy heuristic: if not holding an object, navigate to nearest onion supply; if holding an onion, navigate to nearest available pot. Uses motion-planning via the Overcooked MotionPlanner for pathfinding.
- **Waiter** (deliver): Picks up plates from dispensers, retrieves finished soups from cooking pots, and delivers them to the serving counter. Never picks up raw ingredients or interacts with pots. Heuristic: if not holding, navigate to plate dispenser; if holding plate, navigate to nearest ready soup; if holding soup, navigate to serving counter.
- **Chaos** (obstructive, V2 experiments only): With 30% probability executes INTERACT at the current location; 40% probability takes a random movement action; 30% probability stays in place. Used to test whether agents learn independent strategies under a disruptive partner before converging on the chef/waiter design.

Overcooked variant history. Three variants were developed iteratively. **V1** (greedy/random partner): used built-in GreedyHumanModel and RandomAgent. Failed to produce κ differentiation because both partners require similar optimal strategies from the agent—the policy could ignore partner type and still succeed. **V2** (chef/waiter/chaos): introduced role specialization but included chaos as a “solo” training condition. Failed because chaos was too disruptive (reward=0 prevented any learning), making SINGLE mode κ measurements degenerate. Led to the insight that κ can only be measured when training produces non-zero gradients. **V3** (chef/waiter, static/dynamic): the final design. Removed chaos partner. STATIC mode reveals partner type in observation; DYNAMIC mode hides it and switches roles mid-episode every 30 steps.

κ Computation

Pseudocode

1. **Input:** trained policy π_θ , two identifiable configurations A and B , rollout count N , and discount γ .
2. Initialize $g_A \leftarrow \mathbf{0}$ and $g_B \leftarrow \mathbf{0}$.
3. For $i = 1, \dots, N$, collect $\tau_A \leftarrow \text{Rollout}(\pi_\theta, A)$ and $\tau_B \leftarrow \text{Rollout}(\pi_\theta, B)$, then add the corresponding return-weighted score-function gradients to g_A/N and g_B/N .
4. Compute $g_{\text{mixed}} \leftarrow (g_A + g_B)/2$ and $\mathcal{E} \leftarrow (\|g_A\|^2 + \|g_B\|^2)/2$.
5. If $\mathcal{E} = 0$, return zero energy and $\kappa = N/A$; otherwise return $\kappa = \|g_{\text{mixed}}\|^2/\mathcal{E}$.

Numerical Stability and Edge Cases

κ is computed by:

$$\kappa = \frac{\|(g_A + g_B)/2\|^2}{\max(\|g_A\|^2 + \|g_B\|^2)/2, \varepsilon)} \quad (1)$$

where $\varepsilon = 10^{-10}$ prevents division by zero.

Zero-gradient case. When both g_A and g_B are zero (no reward signal in either configuration), κ is reported as 0.000 (the numerator is zero; the ratio is numerically zero). This occurs in Overcooked DYNAMIC mode, where the agent fails to deliver a single soup—the REINFORCE gradient is zero for all steps, producing $\mathcal{E} = 0$ and $\kappa \rightarrow 0$ by construction. We distinguish this from $\kappa = 0$ due to gradient cancellation ($g_A \approx -g_B$) by reporting “zero energy” separately in the data table.

One-sided energy. When exactly one conditional gradient is non-zero ($\|g_A\| > 0$, $\|g_B\| \approx 0$), algebraically $\kappa = 0.5$. This occurs when the agent learns a strategy effective for one partner but not the other (e.g., Overcooked STATIC seeds where the agent learns to deliver with chef but not to cook with waiter). This produces the same numerical value as genuinely orthogonal gradients, creating an identification ambiguity that should be resolved by inspecting per-configuration rewards (see Section 6).

Sensitivity to N . We use $N = 30$ evaluation rollouts per configuration. At $N = 30$, the standard error of κ across repeated measurements on the same model is < 0.02 for converged policies. Increasing N to 100 changes individual κ estimates by < 0.03 on average.

Gradient Computation per Algorithm Family

Policy-gradient (PPO, A2C, REINFORCE). The REINFORCE-style per-configuration gradient is:

$$g_r = \mathbb{E}_{\tau \sim \pi, r} [\nabla_\theta \log \pi_\theta(a_t | o_t) \cdot R(\tau)] \quad (2)$$

where $R(\tau) = \sum_t \gamma^t r_t$ is the discounted return for trajectory τ under configuration r . Gradients are accumulated across all (o_t, a_t) pairs within the trajectory, weighted by the episode return. This directly measures how the policy’s action distribution *would change* if trained exclusively on data from configuration r .

Value-based (DQN). For DQN, we substitute the TD-loss gradient of the Q-network:

$$g_r = \mathbb{E}_{(o,a,r,o') \sim \mathcal{D}_r} \left[\nabla_{\theta} \frac{1}{2} (Q_{\theta}(o, a) - (r + \gamma \max_{a'} Q_{\theta-}(o', a')))^2 \right] \quad (3)$$

where $\mathcal{D}_r$ is a batch of transitions collected under configuration r and $Q_{\theta-}$ is the target network. This gradient measures how the Q-function’s prediction accuracy would *change* for transitions from configuration r , rather than measuring policy divergence.

Actor-critic (SAC). For SAC, we compute gradients through the *actor network* only, using the soft policy-gradient loss:

$$g_r = \nabla_{\theta} \mathbb{E}_{o \sim \mathcal{D}_r} \left[\sum_a \pi_{\theta}(a|o) (\alpha \log \pi_{\theta}(a|o) - \min_{i=1,2} Q_{\phi_i}(o, a)) \right] \quad (4)$$

This hybrid approach uses Q-values from the critic but the gradient is taken with respect to the actor’s parameters, making it structurally closer to the policy-gradient family than to pure value-based methods.

Why cross-algorithm κ is not comparable in absolute value. Because each algorithm family uses a different gradient field, $\kappa(\text{PPO})$, $\kappa(\text{DQN})$, and $\kappa(\text{SAC})$ measure alignment in *different vector spaces* (policy-gradient space, Q-function space, and soft-actor space, respectively). Proposition 4 below formalizes this: there is no estimator-independent ordering. Consequently, cross-algorithm comparison is restricted to the *directional* signal (whether $\kappa_R > \kappa_H$ or $\kappa_R < \kappa_H$), not the absolute κ magnitude.

Per-Step vs. Per-Episode Reward Weighting

For dense-reward environments (Toy: reward per step), each (o_t, a_t) pair is weighted by the individual step reward r_t . For sparse-reward environments (Overcooked: reward only at soup delivery), all (o_t, a_t) pairs within the episode are weighted by the *episode return* $R(\tau) = \sum_t \gamma^t r_t$, ensuring that early actions contributing to eventual soup delivery receive non-zero gradient weight. For 510K, rewards are terminal (end-of-game scores), so the episode return is used throughout. This design is standard in REINFORCE implementations and ensures κ is computed consistently across diverse reward structures.

Proofs of Propositions

Proposition 1 (Pythagorean Decomposition)

Let $g_A, g_B \in \mathbb{R}^d$ be the policy gradients under configurations A and B . Define $g_+ = \frac{g_A + g_B}{2}$ (shared component) and $g_- = \frac{g_A - g_B}{2}$ (differential component).

$$\begin{aligned} \|g_A\|^2 + \|g_B\|^2 &= \|g_+ + g_-\|^2 + \|g_+ - g_-\|^2 \\ &= (\|g_+\|^2 + \|g_-\|^2 + 2g_+ \cdot g_-) \\ &\quad + (\|g_+\|^2 + \|g_-\|^2 - 2g_+ \cdot g_-) \\ &= 2(\|g_+\|^2 + \|g_-\|^2) \end{aligned} \quad (5)$$

Therefore $\frac{\|g_A\|^2 + \|g_B\|^2}{2} = \|g_+\|^2 + \|g_-\|^2$. Since $\kappa = \|g_+\|^2 / (\|g_+\|^2 + \|g_-\|^2)$, it follows that κ is the fraction of total gradient energy retained after aggregation. The complementary fraction $1 - \kappa = \|g_-\|^2 / (\|g_+\|^2 + \|g_-\|^2)$ is the share of gradient energy *canceled* by hidden information: it is the energy invested in learning incompatible strategies that mutually annihilate during gradient descent.

From this geometric picture we can read off the extreme cases. When $g_A \approx g_B$, then $g_- \approx \mathbf{0}$, $\kappa \approx 1$, and all gradient energy is retained. When $g_A \approx -g_B$, then $g_+ \approx \mathbf{0}$, $\kappa \approx 0$, and the learning signal collapses to zero despite each individual configuration carrying strong gradient energy. The intermediate case $g_A \perp g_B$ yields $g_+ \perp g_-$ with $\|g_+\| = \|g_-\|$, giving $\kappa = 0.5$.

Proposition 2 (Effective Learning Signal)

Under balanced configuration mixing ($p_A = p_B = 1/2$), the expected per-step gradient during training is:

$$\mathbb{E}[g_{\text{train}}] = \frac{1}{2}(g_A + g_B) = g_+ \quad (6)$$

The effective gradient magnitude satisfies:

$$\|g_+\|^2 = \kappa \cdot \frac{\|g_A\|^2 + \|g_B\|^2}{2} \quad (7)$$

When $\kappa \rightarrow 0$, $\|g_+\|^2 \rightarrow 0$ and the policy receives no effective learning signal regardless of the per-configuration gradient magnitudes. During gradient descent, this means update steps shrink toward zero in expectation, producing a stalled optimization trajectory that can be mistaken for convergence.

Proposition 3 (Partition Refinement)

Let a fine relation partition $\mathcal{R}$ refine a coarse partition $\mathcal{C}$, with $g_c = \mathbb{E}[g_r \mid c]$. Both partitions have the same mixed gradient. For every coarse cell c , Jensen’s inequality gives

$$\sum_{r \in c} p(r \mid c) \|g_r\|^2 \geq \left\| \sum_{r \in c} p(r \mid c) g_r \right\|^2 = \|g_c\|^2. \quad (8)$$

Multiplying by p_c and summing over c shows that fine-partition energy is at least coarse-partition energy. Since the retained numerator $\|g_+\|^2$ is unchanged by partition refinement (only the fine/coarse decomposition of the denominator differs), $\kappa_{\mathcal{R}} \leq \kappa_{\mathcal{C}}$ whenever both ratios are defined.

Interpretation. Finer information partitioning always reduces or maintains κ . Adding more hidden configurations never increases gradient retention—it can only reveal more conflict. When a designer splits a single hidden variable into finer sub-categories (e.g., partitioning “teammate” into “teammate-type-A” and “teammate-type-B”), κ is guaranteed not to increase. This proposition provides the formal guarantee behind the continuous reveal curve’s shape: as we progressively add consistent information (thereby coarsening the hidden partition), κ must monotonically increase toward 1.

Proposition 4 (Gradient-Field Specificity)

For policy-gradient methods, g_r^{PG} reflects the direction of improvement in action preferences for configuration r . A TD-loss gradient instead reflects prediction residuals under its data distribution and target.

Consequently there is no estimator-independent ordering between $\kappa(g_A^{\text{PG}}, g_B^{\text{PG}})$ and $\kappa(g_A^{\text{TD}}, g_B^{\text{TD}})$. Changing the update field can change both the angle and the norm ratio of the conditional vectors even for the same environment and policy.

Implication for cross-algorithm comparison. Cross-algorithm κ values must name the estimator; the directional reversal observed in our DQN and SAC experiments is not a proof that either algorithm family is immune to hidden information. Rather, it reveals that the TD-loss and soft-actor gradient fields respond to information structure differently than the REINFORCE gradient field. Because these families use fundamentally different objectives, their κ values inhabit different metric spaces and cannot be directly compared in magnitude. The appropriate cross-algorithm analysis is directional: testing whether the same sign pattern ($\kappa_R > \kappa_H$ or $\kappa_R < \kappa_H$) holds within each family.

Algorithm Families: Information Processing and Behavioral Divergence

The empirical reversal between policy-gradient and value-based methods raises a structural question: *why* do these families respond differently to hidden information? We identify two orthogonal mechanisms—gradient sourcing and experience aggregation—that jointly determine a family’s κ signature.

Mechanism 1: Gradient Sourcing

The source of the gradient fundamentally shapes what κ measures.

Policy-gradient family (PPO, A2C, REINFORCE). The gradient is sourced from the policy’s own action distribution:

$$\nabla_{\theta} \mathcal{L}_{\text{PG}} = \mathbb{E}[\nabla \log \pi_{\theta}(a|o) \cdot A(o, a)],$$

where $A(o, a)$ is an advantage estimate (return, GAE, or TD residual). This gradient *directly encodes* which actions the policy should increase or decrease. When hidden configuration A demands action 0 and configuration B demands action 1, the gradients $\nabla \log \pi(a=0|o)$ and $\nabla \log \pi(a=1|o)$ point in opposite directions—they literally suggest mutually exclusive parameter updates. κ detects this opposition directly: it measures the cosine similarity of action-preference vectors across configurations.

Value-based family (DQN). The gradient is sourced from prediction error minimization:

$$\nabla_{\theta} \mathcal{L}_{\text{TD}} = \mathbb{E}[\nabla_{\theta} Q_{\theta}(o, a) \cdot (Q_{\theta}(o, a) - y)],$$

where $y = r + \gamma \max_{a'} Q_{\theta-}(o', a')$ is the TD target. This gradient updates the Q-function to reduce prediction error—it does not directly encode action preferences. Whether configurations A and B require different optimal actions is irrelevant to the TD gradient; what matters is whether the

Q-function *predicts similar values* for similar states across configurations. Since the transition dynamics of A and B produce similar state distributions in many environments, the Q-function’s prediction task aligns gradients across configurations even when the optimal actions differ.

Consequence: The PG gradient amplifies policy divergence; the TD gradient amplifies state-similarity. This is why κ_{PG} drops when optimal actions diverge, while κ_{TD} can remain high.

Mechanism 2: Experience Aggregation

The replay buffer in off-policy methods introduces a second-order effect.

On-policy (PPO, A2C, REINFORCE). Gradients are computed from a buffer of recent, sequentially collected trajectories. If training alternates between configurations A and B , successive gradient updates pull the policy in opposing directions. The net effect is destructive interference: the policy oscillates, and the effective learning signal shrinks.

Off-policy (DQN, SAC). Gradients are computed from a uniformly sampled replay buffer containing up to 50,000 transitions. If configurations A and B appear with equal frequency, each mini-batch contains a balanced mixture. The average gradient over a batch is approximately $\frac{1}{2}(g_A + g_B)$, which is the *same* regardless of whether g_A and g_B oppose each other. The replay buffer acts as an implicit averager: it pre-averages conflicting signals *before* they reach the optimizer, whereas on-policy methods expose the conflict directly.

Consequence: Larger buffers produce stronger averaging. This predicts that κ_{DQN} should be *higher* (less conflict) as buffer size increases—a prediction consistent with our Toy DQN result ($\kappa_R > \kappa_H$ with a small 2,000-transition buffer) vs. 510K DQN ($\kappa_R < \kappa_H$ with a 50,000-transition buffer).

Combined Effect

The two mechanisms interact multiplicatively:

	Gradient Source	Experience Aggregation
PG	Action-preference	Sequential
Value	Value-prediction	Replay-buffered

Table 1: Two mechanisms determine κ direction.

For PG, action-preference gradients amplify conflict and sequential collection exposes it; for value-based, prediction-error gradients suppress conflict and replay buffers average it.

For policy-gradient methods, both mechanisms push in the same direction: the gradient source amplifies policy divergence, and sequential experience collection exposes it. The net result is κ_{PG} dropping under hidden information.

For value-based methods, the two mechanisms work at cross-purposes: the value-prediction gradient source partially suppresses conflict, while the replay buffer fully averages it. When the buffer is large enough to dominate (510K: 50K transitions), the averaging wins and κ_{TD} actually *increases* under hidden information. When the buffer is small

(Toy: 2K transitions), the gradient source dominates and κ_{TD} decreases—matching the PG pattern.

Practical Implications

1. **Algorithm selection:** In environments where hidden information creates genuine action-preference divergence (Overcooked, Toy), policy-gradient methods provide the cleanest κ signal and the most direct diagnostic. Value-based methods may obscure the conflict behind buffer averaging.
2. **Buffer size as a hyperparameter for κ sensitivity:** Reducing the replay buffer size increases κ sensitivity to hidden information in value-based methods. Researchers using κ diagnostically with off-policy algorithms should consider buffer size when interpreting κ values.
3. **SAC occupies an intermediate position:** SAC’s actor gradient is structurally similar to PG (both operate in action-preference space), but its critic is trained with TD-loss from a replay buffer. The resulting κ is a hybrid—neither fully PG-like nor fully TD-like—consistent with our observation that κ_{SAC} shows a weaker reversal ($\Delta\kappa \approx 0.04$) than κ_{DQN} ($\Delta\kappa \approx 0.12$, but reversed).

Experimental Design

Seed Selection and Independence

All experiments use seeds in the range 41–49 (with Toy experiments also using 0–9 for initial exploratory runs). Seeds are consecutive integers to facilitate reproducibility, with no cherry-picking or post-hoc filtering. Each seed produces an independent training run with different random initialization of network weights, environment state, and stochastic action sampling. No hyperparameter tuning was performed on a per-seed basis; all seeds within an experimental condition share identical hyperparameters.

REINFORCE: Self-Play vs. Single-Agent

Two REINFORCE configurations were tested on 510K:

- **Single-agent (initial):** One agent learns against three random bots. 8 seeds, 5,000–20,000 episodes. High gradient variance due to Monte Carlo estimation without GAE; most seeds failed to converge (reward < 20). Reported κ values reflect non-converged, high-variance policies.
- **Self-play (final):** All four players share the same policy weights, updated from the P0 perspective. 8 seeds, 20,000 episodes. Same conclusion: vanilla REINFORCE cannot stably converge on 510K regardless of the opponent model, confirming that GAE and clipping (afforded by A2C/PPO) are necessary for this task.

The self-play results are reported in the main text and appendix tables. Single-agent results are omitted due to non-convergence but available in the supplementary code.

Training Duration Justification

- **Toy:** 10,000 steps. The toy environment has 20-step episodes, and convergence occurs within $\sim 2,000$ steps

Policy (seed)	Path Length	Endpoint Dist.	Curvature
SINGLE (41)	0.340	0.046	7.3×
SINGLE (51)	0.482	0.068	7.1×
SINGLE (52)	0.561	0.034	16.6×
SINGLE (53)	0.454	0.108	4.2×
SINGLE (54)	0.443	0.051	8.8×
STATIC (41)	0.253	0.071	3.6×
STATIC (61)	0.342	0.086	4.0×
STATIC (62)	0.490	0.073	6.7×
STATIC (63)	0.281	0.048	5.9×
STATIC (64)	0.278	0.021	13.5×
DYNAMIC (41)	0.299	0.046	6.5×
DYNAMIC (42)	0.231	0.112	2.1×
DYNAMIC (43)	0.367	0.026	14.3×
DYNAMIC (44)	0.274	0.031	8.9×
OBVIOUS (71)	0.203	0.100	2.0×
OBVIOUS (72)	0.355	0.122	2.9×
OBVIOUS (73)	0.366	0.051	7.2×
OBVIOUS (74)	0.302	0.022	13.6×
OBVIOUS (75)	0.318	0.036	8.9×
OBVIOUS (76)	0.391	0.085	4.6×
OBVIOUS (77)	0.283	0.043	6.6×
OBVIOUS (78)	0.404	0.035	11.5×

Table 2: Full path integral data for all 22 seeds across 4 modes.

for REVEALED mode. Extended training to 50,000 steps does not change κ beyond statistical noise.

- **510K:** 1,000,000 steps (PPO/A2C/DQN) or 20,000 episodes (REINFORCE). PPO policy-loss curves plateau by $\sim 600K$ steps across all modes. DQN training curves are noisier but batch statistics stabilize within the training budget.
- **Overcooked:** 1,000,000 steps. PPO STATIC reward plateaus by $\sim 500K$ steps. DYNAMIC reward remains at zero throughout ($\kappa = 0$ correspondingly). A2C and DQN fail to produce non-zero reward on this environment at any step count tested.

Statistical Methods

- **Effect sizes:** Cohen’s $d = (\bar{\kappa}_R - \bar{\kappa}_H) / \sigma_{\text{pooled}}$ is reported for representative comparisons.
- **Monotonicity:** Spearman rank correlation between mode ordinality (SINGLE = 1, STATIC = 2, DYNAMIC = 3) and path integral $\mathcal{P}$.
- **Continuous reveal:** One-tailed Welch t -test (unequal variance) comparing κ_p against the $\kappa_{0\%}$ baseline at each partial-information level.
- **Significance threshold:** $p < 0.05$ throughout.

Full Path Integral Data

Env	Alg.	Mode	Retention (mean $\pm$ std)	n
Toy	PPO	HIDDEN	0.039 ± 0.073	10
Toy	PPO	REVEALED	0.726 ± 0.100	10
Toy	A2C	HIDDEN	0.195 ± 0.352	10
Toy	A2C	REVEALED	0.842 ± 0.025	10
Toy	DQN	HIDDEN	0.020 ± 0.021	10
Toy	DQN	REVEALED	0.510 ± 0.017	10
510K	A2C	SINGLE	0.644 ± 0.201	8
510K	A2C	STATIC	0.500 ± 0.000	8
510K	A2C	DYNAMIC	0.519 ± 0.060	8
510K	DQN	SINGLE	0.797 ± 0.123	8
510K	DQN	DYNAMIC	0.917 ± 0.064	8
510K	SAC	SINGLE	0.504 ± 0.038	2
510K	SAC	DYNAMIC	0.540 ± 0.069	2
510K	PPO	SINGLE	0.569	1
510K	PPO	DYNAMIC	0.444 ± 0.069	9
510K	REINFORCE	SINGLE	0.487 ± 0.312	8
510K	REINFORCE	DYNAMIC	0.605 ± 0.238	8
Overcooked	PPO	STATIC	0.500 ± 0.006	8
Overcooked	PPO	DYNAMIC	N/A (zero energy)	8
Overcooked	A2C	STATIC	0.500 ± 0.000	8
Overcooked	A2C	DYNAMIC	0.125 ± 0.217	8
Overcooked	DQN	STATIC	0.473 ± 0.093	8
Overcooked	DQN	DYNAMIC	0.645 ± 0.209	8

Table 3: Complete κ data across environments, algorithms, and modes. Toy DQN ($\kappa_R > \kappa_H$) matches the policy-gradient pattern, indicating the DQN reversal on 510K is driven by replay buffer scale, not an intrinsic property of value-based gradients. Overcooked PPO DYNAMIC has zero conditional-gradient energy (all 8 seeds produce zero reward) so κ is numerically zero.

Extended κ Data

Toy DQN and the reversal pattern. The Toy DQN result ($\kappa_{\text{REVEALED}} = 0.510 > \kappa_{\text{HIDDEN}} = 0.020$) matches the policy-gradient direction, in contrast to 510K DQN ($\kappa_{\text{SINGLE}} = 0.797 < \kappa_{\text{DYNAMIC}} = 0.917$). We attribute this to replay buffer capacity: Toy DQN uses a buffer of 2,000 transitions over 10,000 training steps, averaging only ~ 200 steps of history; 510K DQN uses 50,000 transitions over 1,000,000 steps, averaging $\sim 5,000$ steps. The larger buffer provides more effective mixing of configurations A and B , forcing Q-function gradients into alignment. This supports the interpretation that the DQN reversal is an artifact of buffer scale, not an algorithmic invariant.

Split-Batch Reveal Audit

Training Hyperparameters and Infrastructure

Hardware: NVIDIA GeForce RTX 5070 Ti Laptop GPU (12 GB VRAM), 32 GB RAM. **Software:** Windows 11; PyTorch 2.11; stable-baselines3 2.8; sb3-contrib 2.8; Gymnasium 0.29+; NumPy 2.3. **Training time:** ~ 20 min per PPO seed (8 parallel envs); ~ 2.5 h per SAC seed (sequential); ~ 3.5 min per Toy seed.

Reveal Fraction	Split-batch κ (mean $\pm$ std)	n
0%	0.534 ± 0.038	2
25%	0.446 ± 0.085	2
50%	0.514 ± 0.024	2
75%	0.324 ± 0.079	2
100%	0.616 ± 0.039	2

Table 4: Continuous reveal κ values (510K, PPO, 2 seeds $\times$ 2 rollouts per condition). The W-shape is robust at $n = 2$ seeds per level.

Parameter	Value
Learning rate	3×10^{-4} (PPO/A2C, all envs); 1×10^{-3} (DQN, SAC, REINFORCE)
PPO n_steps	2048 (self-play, 510K) / 256 (parallel, all others)
PPO batch size	64 (self-play) / 256 (parallel)
PPO epochs/update	10
γ (discount)	0.99
GAE λ	0.95
Clip range	0.2
Entropy coeff.	0.01
Network arch.	MLP [256, 256], tanh activation (all envs except Toy: [32, 32])
Parallel envs	8 (PPO/A2C) / 4 (DQN) / 1 (SAC, REINFORCE)
DQN buffer size	50,000 (510K, Overcooked) / 2,000 (Toy)
DQN τ /explore	$\tau = 0.005$; ϵ -greedy $1.0 \rightarrow 0.02$ over 30% of training
SAC α	Auto-tuned: initial 0.2, target entropy $0.98 \cdot \log \mathcal{A} $
REINFORCE lr	1×10^{-3} , batch 10 episodes, 20,000 episodes (self-play)

Table 5: Training hyperparameters for all algorithms and environments.

Code and Data Availability

All source code, experiment configurations, and pre-computed validation results are included in the supplementary material. The codebase includes the 510K game environment with four configurable cooperation modes; the Overcooked role-based partner wrapper (chef, waiter agents); the Toy Matching environment; training scripts for all five algorithms (PPO, A2C, DQN, SAC, REINFORCE) across all three environments; κ computation utilities and the continuous reveal pipeline; and the path integral analysis framework. All materials under MIT License; publicly available upon publication.